CDBench: Controlled Multi-Fault Debugging with Reinforcement Learning and Iterative Repair*

Anonymous

May 18, 2026

Abstract

We study multi-fault debugging under controlled complexity: given source code with N simultaneous bugs injected, can a language model identify and fix all of them? Existing benchmarks fix N=1, making it impossible to measure how capability degrades as complexity scales - or how far sequential, long-horizon reasoning can be pushed before it breaks down. We introduce **CDBench** (Controllable Debugging Benchmark), an initial exploration built on open-source library source files using an adversarial three-player pipeline adapted from the SGS framework [1] that produces 139 verified corruptions. CDBench reveals a consistent degradation curve: Qwen2.5-Coder-32B-Instruct drops from 100% at N=1 to 68% at N=5.

We then ask whether targeted training and inference protocols can close this gap. Applying GRPO reinforcement learning to Qwen2.5-Coder-7B with a functional reward (pytest pass rate), the trained model improves N=3 accuracy from 30% to 47% (+17 pp). **Iterative repair** (re-prompting with failing test names after each round) adds 7–11 pp at N=3–7 and eliminates the degradation cliff; ablations show test names contribute only ∼6 pp while stochastic exploration accounts for most of the gain.

We further identify a **GRPO gradient quality** bottleneck: repair training succeeds only when the model can occasionally generate correct repairs during training rollouts. This provides a principled design criterion for repair-conditioned training in future work.

1 Introduction

Large language models now resolve over 70% of single-bug GitHub issues on SWE-bench Verified [2]. But real debugging rarely involves one bug in isolation. A failing CI pipeline may expose a logic error in a base class surfacing across five callers, a race condition introduced in the same refactor, and an off-by-one error three modules away. The question is not *whether* a model can fix a bug, but *how many simultaneous bugs exhaust its reasoning capacity*—and whether that capacity can be extended through targeted training. Equivalently, N indexes *sequential reasoning depth*: fixing N bugs requires N coordinated locate-and-repair cycles without losing track of prior fixes.

*Code and data: `https://github.com/dinkarjuyal/cdbench`

Existing benchmarks cannot answer this question. SWE-bench, DebugBench [3], and Defects4J [4] all fix N=1. Difficulty is an uncontrolled property of whatever issue happened to exist in the repository. There is no mechanism to measure degradation—how debugging capability changes as a function of compositional complexity.

We introduce **CDBench**, a benchmark that makes bug count N a controllable experimental variable. We inject N simultaneous corruptions into open-source library source code and measure the per-group fix rate as N scales from 1 to 10. Corruptions are generated by instantiating the **SGS** (Self-Guided Self-Play) framework [1] for code: a Proposer LLM suggests single-line mutations, an Executor verifies behavioral change via the test suite, and a Guide LLM filters for non-triviality. This yields 139 verified corruptions across 6 source files. Models output fixes in a compact `FILE/FIND/FIX` format scored by a functional judge running the library's pytest suite.

CDBench reveals a consistent degradation curve. Even Qwen2.5-Coder-32B-Instruct drops from 100% at N=1 to 68% at N=5 and 52% at N=10. All evaluated models degrade monotonically, suggesting sequential task complexity is the binding constraint rather than any single model capability.

RL training closes the gap to 32B with a 7B model. We apply GRPO [5, 6] to Qwen2.5-Coder-7B using pytest pass rate as the reward. The RL-trained model improves N=3 from 30% to 47% (+17 pp vs. few-shot baseline), closing the gap to the 32B ceiling by 27%. At N=5, the model regresses to 16%; sparse reward signal at high N means most GRPO rollouts score 0 and contribute no gradient. Adopting a compact find-replace format is a prerequisite for training: full-file outputs at N=5 exceed the context limit before a fix can be completed, making every rollout score zero regardless of model capability.

Iterative repair flattens the degradation curve. A simple inference-time loop—re-prompt with failing test names after each round—adds 7–11 pp at N=3–7 and produces near-flat accuracy (43–47%) across that range, eliminating the visible degradation cliff. Ablations show that test names add only ∼6 pp; the gain is equivalent to best-of-k sampling: stochastic re-sampling at temperature 0.6 produces diverse attempts, and oracle selection of the best round accounts for most of the improvement. Training stability is gated by whether the model can occasionally succeed during rollouts — a criterion that explains the asymmetric effects across N and points directly to difficulty-filtered training data as the next lever.

Contributions.

1. **CDBench**: this debugging benchmark makes bug count N a controlled experimental variable, enabling empirical measurement of how sequential reasoning depth limits model capability. Built using the SGS corruption generation framework; 139 verified corruptions across scikit-learn. CDBench is extensible: the SGS pipeline generates corruptions for any library with a test suite, and we release tooling to support community additions.
2. **RL for multi-fault debugging**: GRPO with a functional (pytest) reward improves N=3 accuracy by 17 pp (30%→47%), closing 27% of the 7B+fs-to-32B gap. We show

that output format (find-replace vs. full-file) is a prerequisite for training stability, not a hyperparameter.

3. **Iterative repair as inference-time scaling**: re-prompting with failing test names eliminates the degradation cliff at N=3–7. Ablations disentangle test name benefit ($\sim$6 pp) from stochastic exploration (the dominant gain), showing the improvement is equivalent to best-of-k sampling.

2 Related Work

Debugging benchmarks. DebugBench [3] injects single bugs into LeetCode solutions using GPT-4. SWE-bench [2] and agent systems such as SWE-agent [7] and Agentless [8] evaluate models on real GitHub issues, all at N=1. Defects4J [4] and BugsInPy [9] provide curated single-bug collections; Xia et al. [10] survey LLM-based automated program repair (APR), uniformly assuming N=1. Callaghan & Fischer (MSR 2025) mine naturally co-occurring bugs from version history but cannot control bug count or type. CDBench is the first benchmark where N is a controlled variable, enabling degradation curves impossible with prior benchmarks.

RL for code. Traditional APR uses genetic programming [11] or learned templates [12]; LLM-based APR adds test-execution feedback [10]. CodeRL [13] and PPOCoder [14] apply RL with unit-test rewards to code generation. RLEF [15] trains single-bug repair with execution feedback. SWE-RL [16] trains repository-level agents on SWE-bench.

Iterative repair. Self-debugging [17] and self-repair [18] feed execution output back to the model. SelfAPR [19] uses test diagnostics for iterative single-bug repair. ChatRepair [20] iteratively prompts with test feedback for single-bug APR. Iterative repair is also a strategy for extending effective reasoning horizon: each round decomposes a harder task (fix all N bugs) into a shorter one (fix whatever remains), analogous to chain-of-thought scratchpad decomposition of long-horizon tasks. We extend this to multi-fault settings and ablate the contribution of test names vs. stochastic exploration.

3 CDBench Benchmark

3.1 Corruption Generation via SGS

We instantiate the SGS (Self-Guided Self-Play) framework [1] for code corruption generation. The pipeline has three roles:

Proposer. Given a target source file, the Proposer LLM (Qwen2.5-Coder-32B-Instruct) generates candidate single-line mutations: operator swaps ($+\to-$, $\geq \to >$), axis flips (`axis=0`$\to$`axis=1`), set membership inversions (`in`$\to$`not in`), accumulation strategy flips (`+=`$\to$`=`), and similar syntactically-valid one-line errors.

Executor. Each mutation is applied to a sandbox clone. The library's test suite runs; mutations where no test changes status are discarded (no behavioral effect). Surviving mutations define T_fail(c): the set of tests that fail when corruption c is present.

Guide. A second LLM scores each verified mutation on non-triviality (1–5). Mutations scoring ≤ 2 (immediate syntax errors, trivially obvious type mismatches, signature-level detection) are rejected. Mutations scoring ≥ 3 enter the catalog. Appendix A shows accepted and rejected examples with Guide scoring rationale.

Applied to the scikit-learn library, the pipeline produces 173 candidates at a 65% acceptance rate, yielding 139 healthy corruptions after filtering for stable T_fail sets.

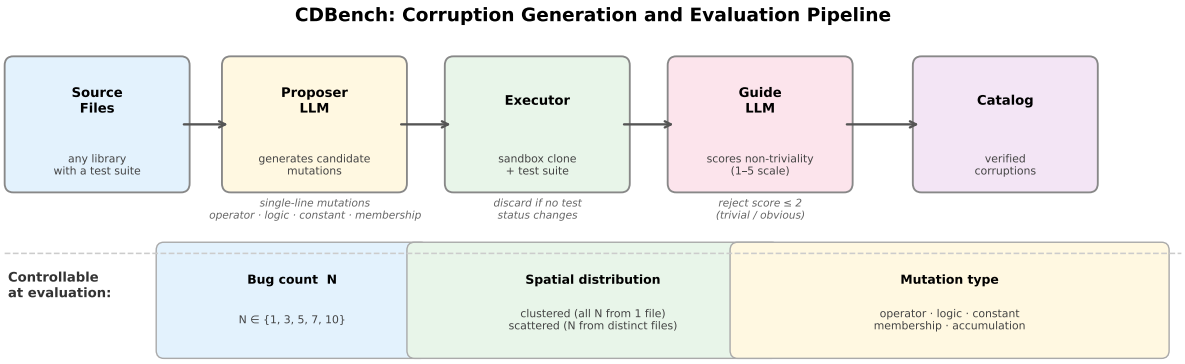

Figure 1: CDBench corruption generation pipeline (top) and the three controllable axes at evaluation time (bottom). The Proposer generates single-line mutations; the Executor filters for behavioral effect via pytest; the Guide rejects trivially-detectable bugs. Bug count N, spatial distribution, and mutation type can be varied independently.

3.2 Evaluation Protocol

Group construction. For each trial, we sample N corruptions uniformly at random from the catalog. We stratify by *spatial distribution*: *clustered* groups draw all N bugs from a single source file—testing whether a model can track multiple simultaneous faults within a single context—while *scattered* groups draw from distinct files, requiring cross-file fault localization. Our hypothesis is: co-located bugs share test coverage and API context, while scattered bugs require the model to reason about disjoint code regions in a single pass. Results aggregate over both distributions unless noted.

Find-replace output format. Prior APR systems use line-level patch formats for single-bug repair [20, 19]. We adapt this to simultaneous N-bug output: models emit one `FILE/FIND/FIX` block per bug:

```
FILE: sklearn/metrics/_classification.py
```

```
FIND: return np.sum(sample_weight[y_true == y_pred])
FIX:  return np.average(sample_weight[y_true == y_pred])
```

Extending this to $N>1$ is non-trivial for training: full-file outputs scale to $\approx$15 000 tokens at $N=5$, causing truncated outputs and zero GRPO gradient from the first training step. The find-replace format reduces output length to 200–400 tokens at $N=5$, eliminating clipping.

Functional scoring. Each `FIND`→`FIX` substitution is applied to the sandboxed library. The functional judge runs T_fail tests for each corruption in the group. Score = fraction of T_fail sets fully passing after repair, averaged across corruptions. This gives per-bug partial credit rather than all-or-nothing group success.

Held-out evaluation. All RL training uses 112 training corruptions. Evaluation uses 27 held-out corruptions, with group composition drawn from this held-out set. This ensures no overlap between training and evaluation groups.

Few-shot prompt. All models receive a 2-bug few-shot (fs) example demonstrating the `FILE/FIND/FIX` format; this abbreviation is used in all table rows (e.g., 7B+fs). This is critical for format compliance at $N>1$: without it, 7B models overwhelmingly emit only one block. We find a scale-dependent sensitivity: few-shot improves 7B by +27 pp at $N=3$ but hurts 3B by -10 pp at $N=1$, suggesting in-context learning capacity for structured format constraints is itself scale-dependent.

4 Experimental Setup

Models. We evaluate Qwen2.5-Coder-Instruct [21] at 3B and 7B (RL training targets) and Qwen2.5-Coder-32B-Instruct (primary ceiling). All inference uses vLLM [22] with temperature 0.6 and top_p= 0.95 for stochastic runs, greedy for round-0 in iterative experiments.

GRPO training. We apply GRPO to Qwen2.5-Coder-7B with: reward = pytest T_fail pass rate (0.0–1.0, functional); $G=8$ rollouts per prompt; LoRA [23] rank 16 on all projection matrices; lr = 10^{-5}, cosine decay over 100 steps; training distribution $N \in \{1, 3, 5\}$ with weights $\{0.5, 0.35, 0.15\}$; 4×H100 GPUs. Training uses only the 112 training corruptions.

Iterative repair. Round 0 uses greedy decoding. Rounds 1–3 use temperature 0.6; the prompt appends the previous `FILE/FIND/FIX` output and the names of still-failing tests. We report *best-of-rounds* (oracle best across all rounds) as the primary metric, and also per-round incremental rates.

Repair-conditioned GRPO. We generate a repair dataset by running the 7B+fs+RL model greedily on 120 training groups (40 per $N \in \{1, 3, 5\}$). Groups where round-0 score < 1.0 become repair examples: the prompt includes the original corrupted code, the model's previous output, and the still-failing test names. We continue training from the 7B+fs+RL checkpoint with a 60/40 mix of standard and repair prompts for 150 steps at lr = 5×10^{-6}.

5 Results

5.1 Degradation Curves

Table 1: Functional debugging accuracy (%, mean ± 95% CI) on CDBench held-out set. 32B: 5 trials per N; others: 10–20 trials. All models use the 2-bug few-shot prompt except 3B/7B base rows.

Model	N=1	N=3	N=5	N=7	N=10
3B (no few-shot)	40±32	17±18	8±10	—	—
3B+fs	30±30	10±20	2±4	—	—
7B (no few-shot)	60±32	3±7	4±5	—	—
7B+fs	70±30	30±27	40±17	—	—
Qwen2.5-Coder-32B+fs	**100±0**	**93±13**	**68±16**	**77±19**	52±26

Figure 2 plots the full degradation curves; Table 1 gives the numeric values with confidence intervals. All models degrade as N increases. The 32B dense model is the strongest baseline, degrading most gracefully. Several findings stand out.

Few-shot size sensitivity. Few-shot prompting has opposite effects at different scales. For 7B, it improves N=3 from 3% to 30% (+27 pp) and N=5 from 4% to 40% (+36 pp). For 3B, it hurts N=1 from 40% to 30% and further worsens multi-fault performance. The 3B model lacks the in-context learning capacity to generalize the format from the demonstration without degrading its base behavior. This implies few-shot prompting must be calibrated per model scale.

5.2 RL Training Results

Table 2: 7B+fs+RL vs. baselines and ceiling. RL improves N=3 by 17 pp (30%→47%), closing 27% of the 7B+fs-to-32B gap. All rows: 20 trials.

Model	N=1	N=3	N=5
7B+fs (baseline)	70%	30%	40%
7B+fs+RL (100 steps)	60%	**47%**	16%
32B+fs (ceiling)	**100%**	**93%**	**68%**
Gap closed (7B+fs→32B)	—	27%	—

At N=3, the RL-trained model closes 27% of the gap from the 7B+fs baseline to the 32B ceiling (30%→47%, +17 pp; 63 pp gap reduced to 46 pp remaining). The N=5 regression (40%→16%) reflects training distribution dynamics: N=5 groups have sparse reward during training (most rollouts score 0), contributing little gradient, while the model's behavior shifts toward patterns effective at N=1, 3.

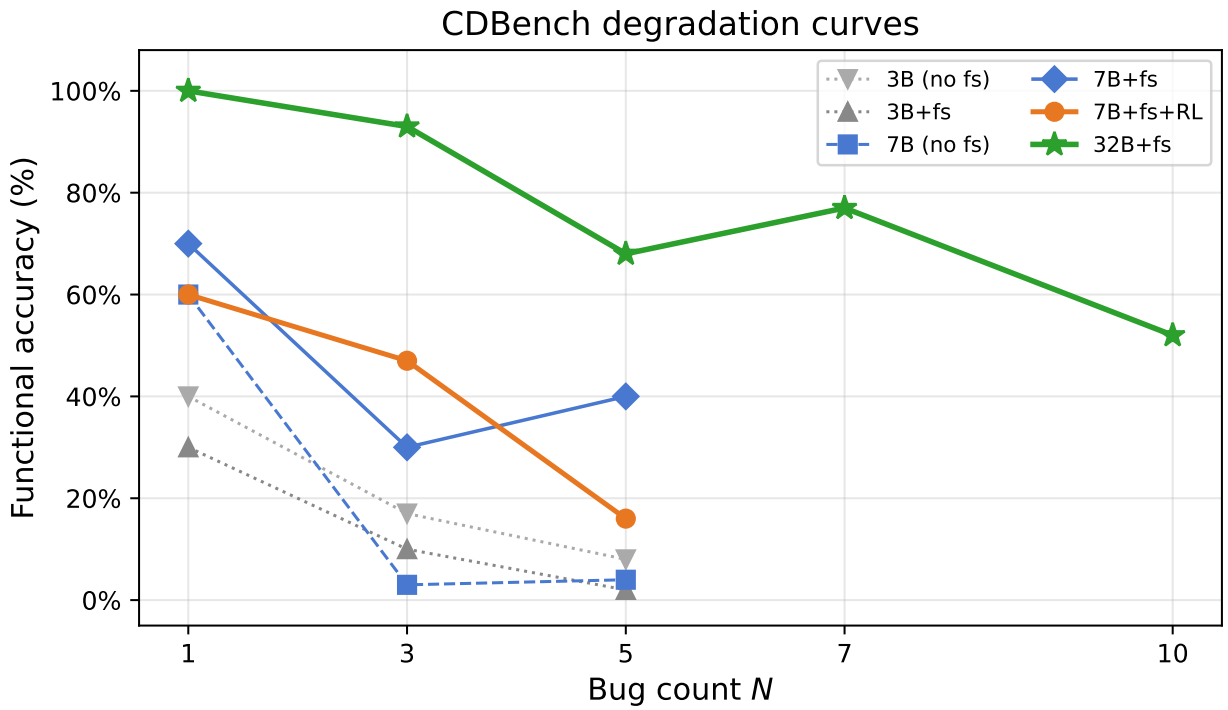

Figure 2: Functional debugging accuracy vs. bug count N. All models degrade monotonically; the 32B dense model degrades most gracefully, from 100% at N=1 to 52% at N=10. Smaller models are evaluated at $N \in \{1, 3, 5\}$ only; performance had already collapsed to near-zero at N=5, making higher N uninformative for those models.

The 3B model with RL (no few-shot, balanced curriculum) improves N=1 from 40% to 70% (+30 pp) with no regression at N=3, reaching parity with the untuned 7B baseline. Few-shot RL training for 3B fails: the few-shot prompt causes the model to generate 8K+ token outputs (over 80% of rollouts hit the token ceiling), indicating the 3B lacks capacity to follow format constraints from the demonstration under RL generation pressure.

5.3 Iterative Repair and Repair-Conditioned Training

Table 3: Iterative repair: round-0 vs. best-of-3-rounds, and test-name ablation (full feedback vs. previous attempt only). 20 trials per N.

	N=3		N=5		N=7	
	Round 0	Best	Round 0	Best	Round 0	Best
7B+fs+RL (full feedback)	40%	47%	32%	43%	33%	44%
7B+fs+RL (prev only)	40%	43%	32%	43%	33%	41%
32B+fs	93%	93%	68%	68%	77%	77%

Iterative repair adds 7–11 pp at N=3–7, producing near-flat accuracy (43–47%) across that

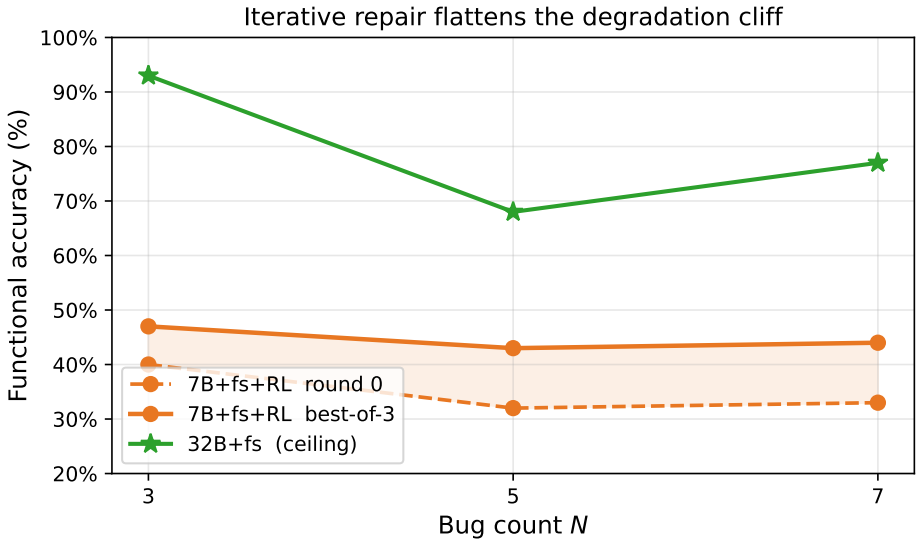

Figure 3: Iterative repair (best-of-3 rounds) produces near-flat accuracy across N=3–7, eliminating the degradation cliff visible in the round-0 curve. The shaded region shows the gain from stochastic re-sampling; the 32B ceiling is shown for reference.

range. Repair success saturates after round 1: hard groups remain hard across all subsequent attempts. Test names contribute only ∼6 pp at N=3 and nothing at N=5; the dominant gain is stochastic exploration at temperature 0.6. Best-of-rounds is therefore equivalent to best-of-k sampling—the gain could be realized via parallel sampling without a sequential feedback loop.

Table 4: Repair-conditioned training vs. 7B+fs+RL baseline. Round-0 and best-of-3-rounds accuracy. 20 trials per N.

	N=1		N=3		N=5	
	Round 0	Best	Round 0	Best	Round 0	Best
7B+fs+RL (baseline)	60%	60%	40%	47%	32%	43%
7B+fs+RL+repair	50%	50%	**58%**	**60%**	33%	**46%**
Δ vs. baseline	−10 pp	0	+18 pp	+13 pp	+1 pp	+3 pp

Repair-conditioned training produces asymmetric gains: +18 pp at N=3 (single-shot, before any repair context), +1 pp at N=5, and −10 pp at N=1. The asymmetry follows from GRPO gradient quality. The repair dataset consists of past failures, and the difficulty of those failures varies sharply: N=1 examples are complete failures (0 remaining bugs fixed), N=3 examples average 2.25 remaining bugs, and N=5 examples average 3.6. GRPO requires nonzero reward variance across its G=8 rollouts to produce a gradient; when the model almost never succeeds on a repair example, all rollouts return the same score and no learning occurs. N=3 repair examples sit in the tractable regime—the model occasionally completes them, gradient flows, and learned priors transfer to single-shot N=3 performance. N=5

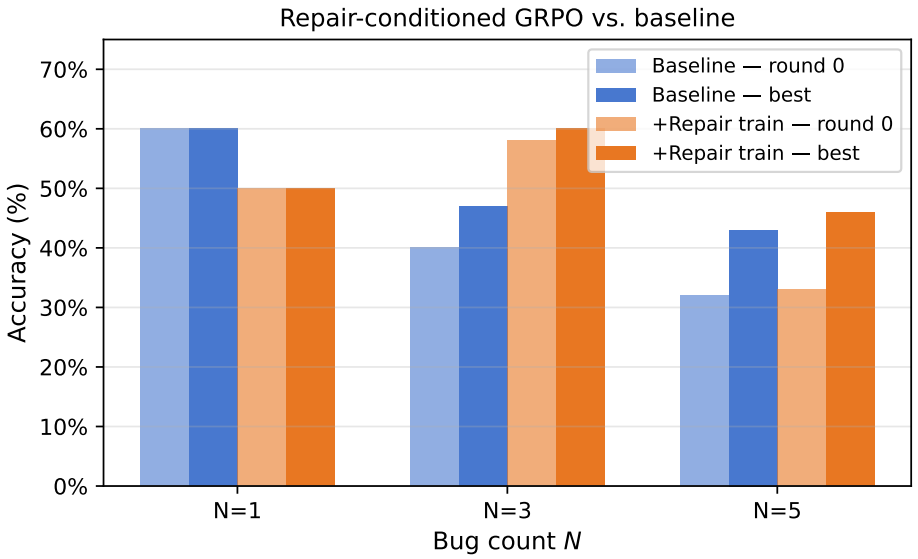

Figure 4: Repair-conditioned GRPO (7B+fs+RL+repair) vs. baseline (7B+fs+RL). N=3 gains +18 pp single-shot from tractable repair examples; N=5 gains only in iterative rounds via format transfer; N=1 regresses due to distribution shift toward hard failures.

repair examples are too hard; the +3 pp iterative gain there comes from format transfer learned on N=3 examples, not from N=5-specific gradient. This has a design implication: repair examples must be sampled from the difficulty regime where the model can occasionally succeed.

5.4 Cross-Library Transfer

To test whether RL-trained debugging behavior generalizes across domains, we evaluate the sklearn-trained 7B+fs+RL model on 30 SciPy corruptions (from `scipy.stats` and `scipy.optimize`) without any SciPy-specific RL training. We compare against a 7B+fs baseline and a 32B+fs ceiling, each evaluated on the same pool.

Table 5: Zero-shot transfer to SciPy. The 7B+RL model was trained exclusively on scikit-learn; no SciPy-specific training was performed. Metric: mean fraction of bugs fixed per trial (same partial-credit metric as Table 1). 7B models: 40 trials per N; 32B: 20 trials.

Model	N=1	N=3	N=5
7B+fs (baseline)	35%	62%	56%
7B+fs+RL (sklearn-trained)	**43%**	52%	47%
32B+fs (ceiling)	**75%**	**68%**	**63%**

RL training on scikit-learn partially transfers to SciPy: N=1 improves from 35% to 43% (+7.5 pp), indicating that the basic single-bug debugging prior generalizes across library domains. However, the N=3 benefit does not transfer—the RL model regresses to 52%

(from the 62% baseline, −10 pp), widening rather than closing the gap to the 32B ceiling at that fault count. This suggests the RL model learned scikit-learn-specific bug patterns during training rather than a general multi-fault strategy. Inspection of the training corpus suggests a mechanism: 54% of the 112 training corruptions are simple operator or constant perturbations (`comparison_flip`, `arith_operator`, `logic_operator`, `constant_change`), and RL rewards this fix vocabulary heavily. SciPy bugs involve similar surface-level patterns but with different domain semantics (library-specific variable names, array conventions, numerical thresholds). The RL model applies its learned sklearn fix vocabulary in the wrong contexts: outputs on SciPy show confident comparison flips and operator changes even when the fix requires different domain knowledge, which degrades multi-fault performance where compound errors amplify each mismatch.

The 32B ceiling itself drops substantially from scikit-learn (N=1: 100% → 75%) to SciPy, confirming that SciPy corruptions present distinct challenges even for strong baselines. Three questions follow for future work. First, *near-vs-far domain transfer*: scikit-learn and SciPy share NumPy array conventions and similar test structures; a near-far spectrum could test whether transfer degrades continuously with domain distance (scikit-learn → SciPy → Django → embedded C). Second, *multi-domain training*: mixing scikit-learn and SciPy corruptions during RL may improve transfer without sacrificing in-domain performance. Third, *domain-adaptive reward*: a library-agnostic reward signal (e.g., test coverage delta rather than library-specific pytest pass rate) may produce more transferable policies.

Limitations and open questions. The held-out evaluation set is 27 corruptions, which limits statistical power; the wide confidence intervals in Table 1 reflect this, and results should be interpreted as directional rather than definitive. Scaling the benchmark—more corruptions, more libraries—is the clearest path to higher-confidence claims, and is the motivation for releasing CDBench as extensible infrastructure. The iterative repair oracle (best-of-rounds) requires knowing which round was best at evaluation time; a deployed system would need a stopping criterion or a verifier. Finally, all RL experiments use a single model family (Qwen2.5-Coder) and a single domain (scikit-learn); generalization of the training recipe to other architectures and domains is assumed but not demonstrated.

6 Conclusion

CDBench makes bug count a tunable proxy for sequential reasoning depth. Key findings: (1) all models degrade monotonically with N—even the 32B ceiling drops from 100% at N=1 to 52% at N=10, suggesting sequential task complexity is the binding constraint; (2) GRPO with functional reward improves N=3 by 17 pp, closing 27% of the 7B+fs-to-32B gap; (3) iterative repair is equivalent to best-of-k sampling, eliminating the degradation cliff at N=3–7; (4) repair-conditioned training requires tractable repair examples to produce gradient—this criterion explains the asymmetric gains across N; (5) zero-shot transfer to SciPy shows the N=1 gain generalizes but the N=3 gain does not, pointing to multi-domain training as a necessary next step.

Format design, inference-time compute, and training signal tractability are more predictive of multi-fault debugging performance than raw model scale. CDBench is released as extensible

infrastructure: the SGS pipeline generates corruptions for any library with a test suite.

Acknowledgements

The author thanks Chandan Akiti for providing compute resources and for helpful discussions that shaped several of the ideas in this paper.

References

[1] Scaling self-play with self-guidance. *arXiv preprint arXiv:2604.20209*, 2025.

[2] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[3] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. DebugBench: Evaluating debugging capability of large language models. *arXiv preprint arXiv:2401.04621*, 2024.

[4] René Just, Darioush Jalali, and Michael D Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pages 437–440, 2014.

[5] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxian Song, Mingchuan Zhang, Yuanchen Li, Yu Wu, Daya Guo, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[6] Daya Guo, Dejian Yang, Haowei Zhang, Junxian Song, Ruoyu Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[7] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[8] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[9] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Etienne, Shantnu Sharma, et al. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1556–1560, 2020.

[10] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. *Proceedings of the 45th International Conference on Software Engineering*, 2023.

[11] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. In *IEEE Transactions on Software Engineering*, volume 38, 2012.

[12] Fan Long and Martin Rinard. Automatic patch generation by learning correct code. In *ACM SIGPLAN Notices*, volume 51, 2016.

[13] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[14] Mohammadali Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. PPOCoder: Execution-based code generation using proximal policy optimization. In *Findings of the Association for Computational Linguistics: EACL 2023*, 2023.

[15] Jonas Gehring, Kunhao Zheng, Jade Copet, Quentin Carbonneaux, Daniel Cohen, Gabriel Synnaeve, and Manmohan Chandraker. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.

[16] Yuxiang Wei, Rishi Bhatt, Matthijs Douze, Tariq Gupta, et al. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[17] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.

[18] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *arXiv preprint arXiv:2306.09896*, 2023.

[19] He Ye, Matias Martinez, and Martin Monperrus. SelfAPR: Self-supervised program repair with test execution diagnostics. In *37th IEEE/ACM International Conference on Automated Software Engineering*, 2022.

[20] Chunqiu Steven Xia and Lingming Zhang. ChatRepair: A conversational approach for automatic program repair. In *ACM International Conference on the Foundations of Software Engineering*, 2024.

[21] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[23] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

A Corruption Examples

The following examples are drawn from the CDBench catalog. Each entry shows the source file path, the single-line diff applied by the Proposer, and a brief note on why the mutation is (or is not) non-trivial. Examples A.1–A.2 passed both the Executor filter and Guide scoring (≥ 3); Example A.3 passed the Executor but was rejected by the Guide.

A.1 Operator Mutation — accepted (Guide score: 4)

File: `sklearn/linear_model/_logistic.py`

```
# convergence check
if n_iter_ >= max_iter:
if n_iter_ > max_iter:
    warnings.warn("lbfgs failed to converge", ConvergenceWarning)
```

The off-by-one shifts the convergence warning by exactly one iteration. Tests that assert `ConvergenceWarning` is raised on tight iteration budgets now silently pass; fixing it requires tracing the solver loop rather than inspecting the warning call site. Guide reasoning: *"The mutated condition differs by a single character with identical control flow on all but the boundary case; a reader must check the loop counter semantics to identify the fault. Score: 4."*

A.2 Membership Mutation — accepted (Guide score: 3)

File: `sklearn/ensemble/_forest.py`

```
# validate max_features string argument
if self.max_features not in ("sqrt", "log2", "auto"):
if self.max_features in ("sqrt", "log2", "auto"):
    raise ValueError(
        f"Invalid max_features: {self.max_features!r}")
```

Inverting the membership test causes validation to reject every valid string argument while silently accepting arbitrary invalid ones. The symptom is a `ValueError` on correct calls rather than incorrect ones—misleading because the error message names a valid value. Guide reasoning: *"The logic inversion is confined to one line, but the symptom (valid input rejected, invalid input accepted) is non-obvious from the error message alone. Score: 3."*

A.3 Trivially Detectable Mutation — rejected (Guide score: 2)

File: `sklearn/utils/validation.py`

```
def check_is_fitted(estimator):
    if estimator is None:
    if estimator is not None:
        raise NotFittedError(
            f"{type(estimator).__name__} is not fitted yet.")
```

This passed the Executor filter because every test that passes a valid fitted estimator now raises `NotFittedError`. The Guide rejected it (score: 2) with the reasoning: *"The inverted None check fires immediately on any valid estimator; a developer reading one line of context identifies the fault without executing the test suite. Score: 2."*